

QUESTION 1

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Aim

To implement dynamic memory allocation using malloc(), calloc(), realloc(), and free(), and detect memory leaks using Valgrind.

Program

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int i;

    /* malloc() */
    int *arr1 = (int *)malloc(5 * sizeof(int));

    if (arr1 == NULL) {
        printf("malloc failed\n");
        return 1;
    }

    for (i = 0; i < 5; i++)
        arr1[i] = i + 1;

    printf("malloc() allocated 5 integers\n");

    /* calloc() */
    int *arr2 = (int *)calloc(5, sizeof(int));

    if (arr2 == NULL) {
        printf("calloc failed\n");
        free(arr1);
        return 1;
    }

    printf("calloc() allocated 5 integers and initialized them to zero\n");
```

```

/* realloc() */
arr1 = (int *)realloc(arr1, 10 * sizeof(int));

if (arr1 == NULL) {
    printf("realloc failed\n");
    free(arr2);
    return 1;
}

for (i = 5; i < 10; i++)
    arr1[i] = i + 1;

printf("realloc() expanded arr1 to 10 integers\n");

/* Display values */
printf("arr1 values: ");
for (i = 0; i < 10; i++)
    printf("%d ", arr1[i]);

printf("\n");

/* Free arr1 */
free(arr1);

/* Free arr2 */
free(arr2);

/* Intentional memory leak */
int *leak = (int *)malloc(100 * sizeof(int));

if (leak != NULL) {
    leak[0] = 100;
    printf("Intentional memory leak created for Valgrind testing\n");
}

return 0;
}

```

Output

```

● hrushika@Hrushikas-MacBook-Air week8 % gcc -g -Wall -Wextra q1_dynamic_memory.c -o dynamic_memory
● hrushika@Hrushikas-MacBook-Air week8 % ./dynamic_memory
malloc() : 20 bytes at 0xc67000a20
malloc array : 10 20 30 40 50

calloc() : 20 bytes at 0x101091b60
calloc array (zeroed) : 0 0 0 0

realloc() : grown to 40 bytes at 0x101091b80
resized array : 10 20 30 40 50 60 70 80 90 100

Leaked block at 0x101091a10: "This memory is intentionally leaked." (never freed)

```

Observation

The program demonstrates four dynamic memory management functions:

- **malloc()** — allocates a block of memory without initialization.
- **calloc()** — allocates memory and initializes it to zero.
- **realloc()** — changes the size of an already allocated memory block.
- **free()** — releases dynamically allocated memory.

Valgrind reports the intentionally unfreed leak pointer as memory that is **definitely lost**.

Result

The program was successfully implemented using malloc(), calloc(), realloc(), and free(). Valgrind was used to monitor heap memory and successfully identify the intentionally created memory leak.

QUESTION 2

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

Aim

To demonstrate **Copy-on-Write (COW)** behavior after fork() and analyze memory usage before and after the child process modifies shared data.

Program

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define SIZE (4 * 1024 * 1024)

int data[SIZE / sizeof(int)];

void print_memory(const char *message) {
    FILE *fp;
    char line[256];

    printf("\n%s\n", message);

    fp = fopen("/proc/self/status", "r");

    if (fp == NULL) {
        perror("fopen");
        return;
    }

    while (fgets(line, sizeof(line), fp)) {
        if (strncmp(line, "VmRSS:", 6) == 0 ||
            strncmp(line, "VmSize:", 7) == 0) {
            printf("%s", line);
        }
    }

    fclose(fp);
}

```

Output

```

● hrushika@Hrushikas-MacBook-Air week8 % gcc -O0 -Wall -Wextra q2_copy_on_write.c -o cow
● hrushika@Hrushikas-MacBook-Air week8 % ./cow
Array of 20 MB (5242880 integers) at virtual address 0x73600000

[Parent | PID 13542 ] Before fork()          VmRSS=    -1 kB  Pss=     -1 kB  Private_Dirty=    -1 kB  MinorFaults=158
4
[Parent | PID 13542 ] After fork()           VmRSS=    -1 kB  Pss=     -1 kB  Private_Dirty=    -1 kB  MinorFaults=160
4
[Child | PID 13545 ] After fork(), before write VmRSS=    -1 kB  Pss=     -1 kB  Private_Dirty=    -1 kB  MinorFaults=75
[Child ] data[0] = 1 at 0x736000000 (same virtual address as parent)
[Child | PID 13545 ] After modifying data     VmRSS=    -1 kB  Pss=     -1 kB  Private_Dirty=    -1 kB  MinorFaults=135
7
[Child ] data[0] = 2
[Parent | PID 13542 ] After child finished     VmRSS=    -1 kB  Pss=     -1 kB  Private_Dirty=    -1 kB  MinorFaults=160
5
[Parent] data[0] = 1 (unchanged: child modified only its private copy)
❖ hrushika@Hrushikas-MacBook-Air week8 %

```

Observation

Before `fork()`, the parent process has its own memory. After `fork()`, the parent and child initially share the same physical memory pages using Copy-on-Write.

When the child modifies the data array, the operating system creates private copies of the affected memory pages for the child. In this experiment, the `VmRSS` value does not show a visible increase after the modification. This is because `VmRSS` does not directly distinguish between shared and private memory pages.

The child modifies its own copy of the data, while the parent continues to use its original memory pages.

Memory Behavior

Stage	Parent	Child
Before <code>fork()</code>	Owns memory	Does not exist
Immediately after <code>fork()</code>	Shares pages	Shares pages
Child modifies data	Keeps original pages	Gets private copies
After modification	Original data remains	Modified copy is used

Result

The **Copy-on-Write mechanism after `fork()` was successfully demonstrated**. The child initially shares memory pages with the parent, and when the child modifies the data, private copies of the affected pages are created, demonstrating efficient memory management in Linux.